

CS 352

Computer Graphics

Lecture 5

Design of Circle Algorithms

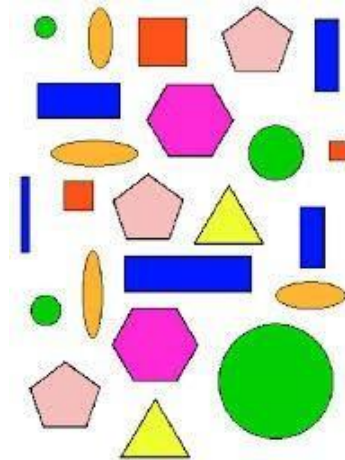


Output primitives

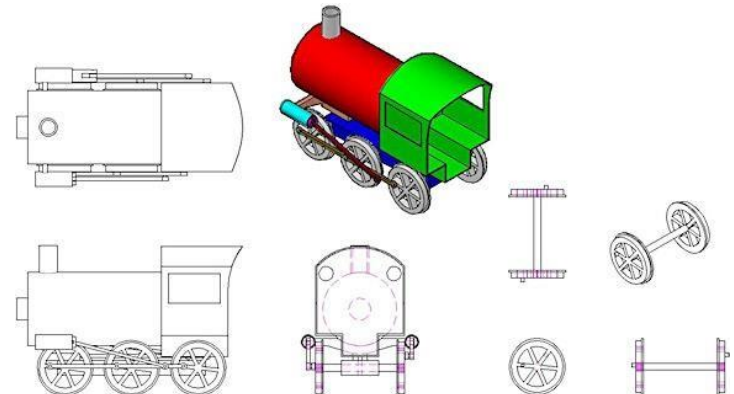
- Graphics programming packages provide functions to describe a scene in terms of basic geometric structures referred to as Output Primitives

- Output primitives include:**

- Points
- Lines
- Circles
- Spline curves



- Output primitives can be used to present more complex objects**

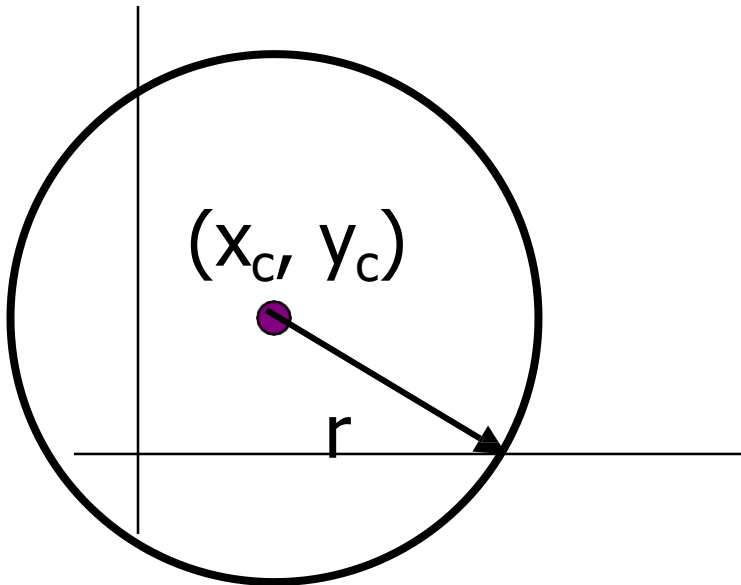




2- Circle Drawing Algorithm

- A. A- Simple Circle Drawing Algorithm : *naïve* algorithm
- B. Simple Circle drawing Algorithm using polar coordinates
- C. Circle Drawing Algorithm using eight-way symmetry
- D. Midpoint Circle Algorithm

A- Simple Circle Drawing Algorithm: *naïve* algorithm



$$(x - x_c)^2 + (y - y_c)^2 = r^2$$

or

$$y = y_c \pm \sqrt{r^2 - (x - x_c)^2}$$



A- Simple Circle Drawing Algorithm: *naïve* algorithm

- The equation for a circle is:

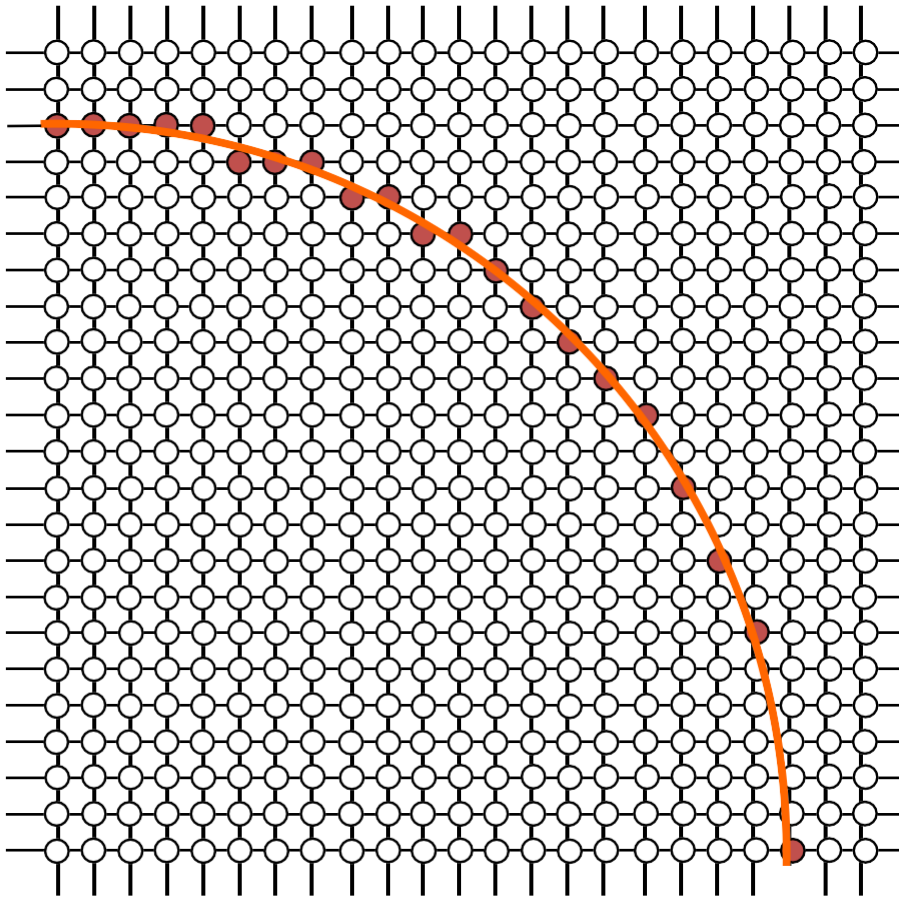
$$x^2 + y^2 = r^2$$

- where r is the radius of the circle
- So, we can write a simple circle drawing algorithm by solving the equation for y at unit x intervals using:

$$y = \pm \sqrt{r^2 - x^2}$$



A- Simple Circle Drawing Algorithm: *naïve* algorithm



$$y_0 = \sqrt{20^2 - 0^2} \approx 20$$

$$y_1 = \sqrt{20^2 - 1^2} \approx 20$$

$$y_2 = \sqrt{20^2 - 2^2} \approx 20$$

⋮

$$y_{19} = \sqrt{20^2 - 19^2} \approx 6$$

$$y_{20} = \sqrt{20^2 - 20^2} \approx 0$$



A- Simple Circle Drawing Algorithm: *naïve* algorithm

- Circle equation: $x^2 + y^2 - r^2 = 0$
- Simple algorithm:

```
for x = xmin to xmax
    y = sqrt(r*r - x*x)
    draw pixel(x, y)
```
- Work by octants and use symmetry
- However, unsurprisingly this is not a brilliant solution!
- Firstly, the resulting circle has large gaps where the slope approaches the vertical
- Secondly, the calculations are not very efficient
 - The square (multiply) operations
 - The square root operation – try really hard to avoid these!
- We need a more efficient, more accurate solution



B- Simple Circle drawing Algorithm using polar coordinates

A better approach, which is still inefficient but avoids the gaps is to plot

$$x = x_c + r \cos \theta$$

$$y = y_c + r \sin \theta$$

as θ takes on values between 0 and 360 degrees.

Time consuming due to trigonometric computations.



C- Circle Drawing Algorithm using eight-way symmetry

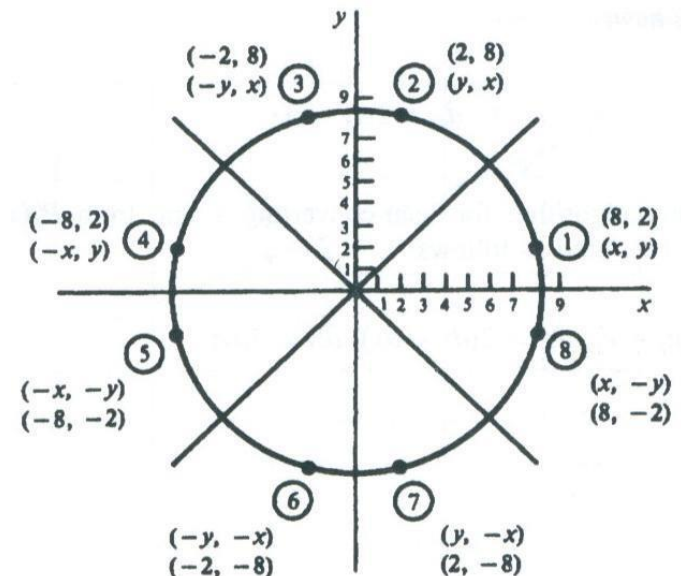
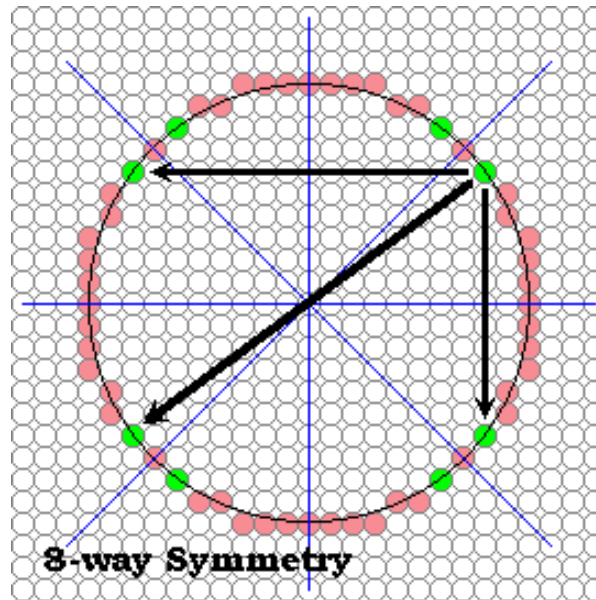
- We only need to calculate the values on the border of the circle in the first octant. The other values may be determined by symmetry.
- **Computation can be reduced by considering the symmetry of circles**

Procedure Circle_Points(x,y :Integer);

Begin

```
Plot(x,y);  
Plot(y,x);  
Plot(y,-x);  
Plot(x,-y);  
Plot(-x,-y);  
Plot(-y,-x);  
Plot(-y,x);  
Plot(-x,y)
```

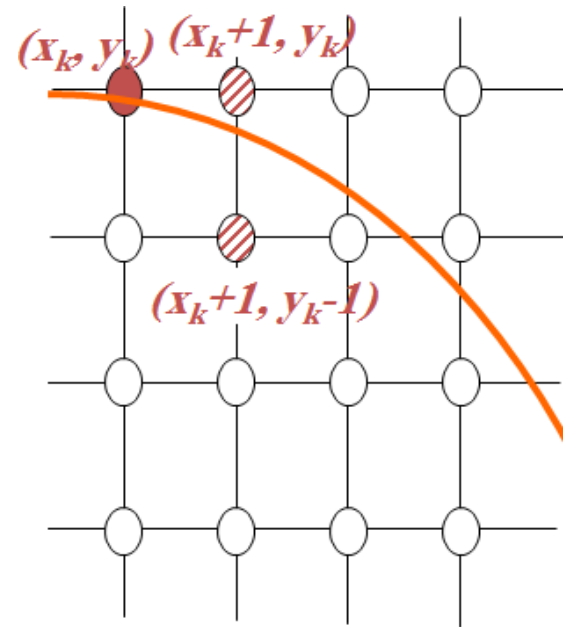
End;





D- Midpoint Circle Algorithm Derivation

- Similarly to the case with lines, there is an incremental algorithm for drawing circles – the *mid-point circle algorithm*
- In the mid-point circle algorithm we use eight-way symmetry so only ever calculate the points for the top right eighth of a circle, and then use symmetry to get the rest of the points
- Consider only the first octant of a circle of radius r centered on the origin. We begin by plotting point $(r, 0)$ and end when $x < y$.
- Assume that we just plotted point (x_k, y_k)
- The next point is a choice between (x_k+1, y_k) and (x_k+1, y_k-1)
- We would like to choose the point that is nearest to the actual circle
- So how do we make this choice?





D- Midpoint Circle Algorithm Derivation

$$f(x,y) = x^2 + y^2 - r^2 = 0$$

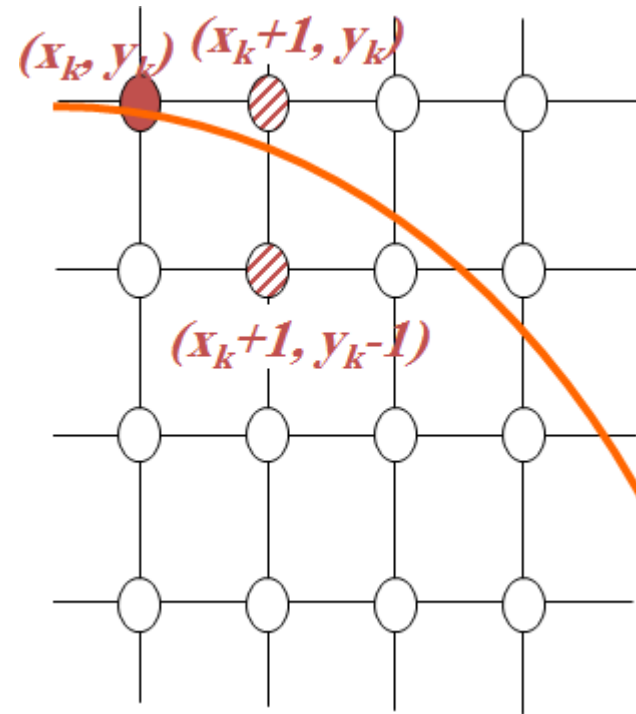
$$p_k = f_{circle}\left(x_k + 1, y_k - \frac{1}{2}\right)$$

$$= (x_k + 1)^2 + \left(y_k - \frac{1}{2}\right)^2 - r^2$$

$$= x_k^2 + 2x_k + 1 + y_k^2 - y_k + \frac{1}{4} - r^2$$

If $p_k < 0$: Plot $(x_k + 1, y_k)$

If $p_k \geq 0$: Plot $(x_k + 1, y_k - 1)$





D- Midpoint Circle Algorithm Derivation

$$\begin{aligned} p_{k+1} &=_{\text{circle}} \left(x_{k+1} + 1, y_{k+1} - \frac{1}{2} \right) \\ &= (x_k + 1 + 1)^2 + \left(y_{k+1} - \frac{1}{2} \right)^2 - r^2 \\ &= x_k^2 + 4x_k + 4 + y_{k+1}^2 - y_{k+1} + \frac{1}{4} - r^2 \end{aligned}$$



D- Midpoint Circle Algorithm Derivation

$$p_k = x_k^2 + 2x_k + 1 + y_{k+1}^2 - y_k + \frac{1}{4} - r^2$$

$$p_{k+1} = x_k^2 + 4x_k + 4 + y_{k+1}^2 - y_{k+1} + \frac{1}{4} - r^2$$

$$p_{k+1} = p_k + (2x_k + 2) + 1 + (y_{k+1}^2 - y_k^2) - (y_{k+1} - y_k)$$

$$p_{k+1} = p_k + 2x_{k+1} + 1 + (y_{k+1}^2 - y_k^2) - (y_{k+1} - y_k)$$



If $y_{k+1} = y_k$,

then:

$$(y_k^2 - y_k^2) - (y_k - y_k) = 0$$

If $y_{k+1} = y_k - 1$,

then:

$$\begin{aligned} & \left((y_k - 1)^2 - y_k^2 \right) - \left((y_k - 1) - y_k \right) = \\ & \left(y_{k_2}^2 - 2y_k + 1 - y_{k_2}^2 \right) - (-1) = \end{aligned}$$

$$-2y_k + 2 = -2(y_k$$



D- Midpoint Circle Algorithm Derivation

$$(x_0, y_0) = (0, r)$$

$$p_k = f_{circle}\left(x_k + 1, y_k - \frac{1}{2}\right)$$

$$p_0 = f_{circle}\left(0 + 1, r - \frac{1}{2}\right)$$

$$= 1 + \left(r - \frac{1}{2}\right)^2 - r^2$$

$$= 1 + r^2 - r + \frac{1}{4} - r^2$$

$$= \frac{5}{4} - r \approx 1 - r$$



D- Midpoint Circle Algorithm Derivation

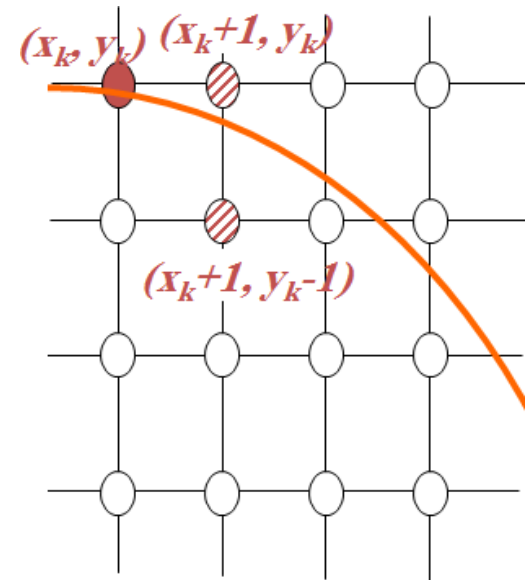
At each point:

If $p_k < 0$: Plot $(x_k + 1, y_k)$

$$p_{k+1} = p_k + 2(x_k + 1) + 1$$

If $p_k \geq 0$: Plot $(x_k + 1, y_k - 1)$

$$p_{k+1} = p_k + 2(x_k + 1) + 1 - 2(y_k - 1)$$





D- Midpoint Circle Algorithm

Midpoint Circle Algorithm

1. Input radius r and circle center (x_c, y_c) , and obtain the first point on the circumference of a circle centered on the origin as

$$(x_0, y_0) = (0, r)$$

2. Calculate the initial value of the decision parameter as

$$p_0 = \frac{5}{4} - r$$

3. At each x_k position, starting at $k = 0$, perform the following test: If $p_k < 0$, the next point along the circle centered on $(0, 0)$ is (x_{k+1}, y_k) and

$$p_{k+1} = p_k + 2x_{k+1} + 1$$

Otherwise, the next point along the circle is $(x_k + 1, y_k - 1)$ and

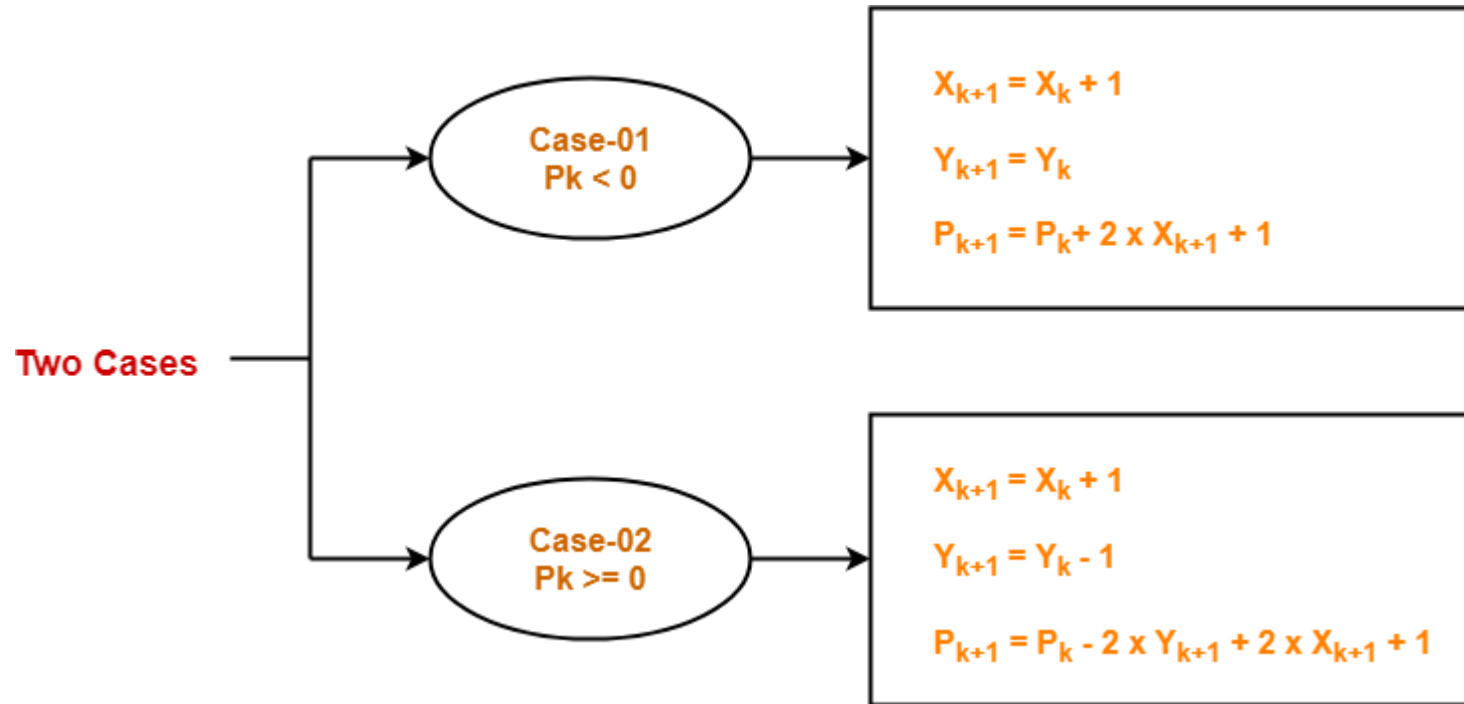
$$p_{k+1} = p_k + 2x_{k+1} + 1 - 2y_{k+1}$$

where $2x_{k+1} = 2x_k + 2$ and $2y_{k+1} = 2y_k - 2$.

4. Determine symmetry points in the other seven octants.
5. Move each calculated pixel position (x, y) onto the circular path centered on (x_c, y_c) and plot the coordinate values:

$$x = x + x_c \quad y = y + y_c$$

6. Repeat steps 3 through 5 until $x \geq y$.



Example 1: Given the center point coordinates (0, 0) and radius as 10, generate all the points to form a circle.

Step-1:

Assign the starting point coordinates (X0, Y0) as-

$$X0 = 0$$

$$Y0 = R = 10$$

Step-2:

Calculate the value of initial decision parameter P0 as-

$$P0 = 1 - R$$

$$P0 = 1 - 10$$

$$P0 = -9$$

Step-3:

As $P0 < 0$, so case-01 is satisfied.

Thus,

$$X1 = X0 + 1 = 0 + 1 = 1$$

$$Y1 = Y0 = 10$$

$$P1 = P0 + 2 X1 + 1 = -9 + (2 \times 1) + 1 = -6$$

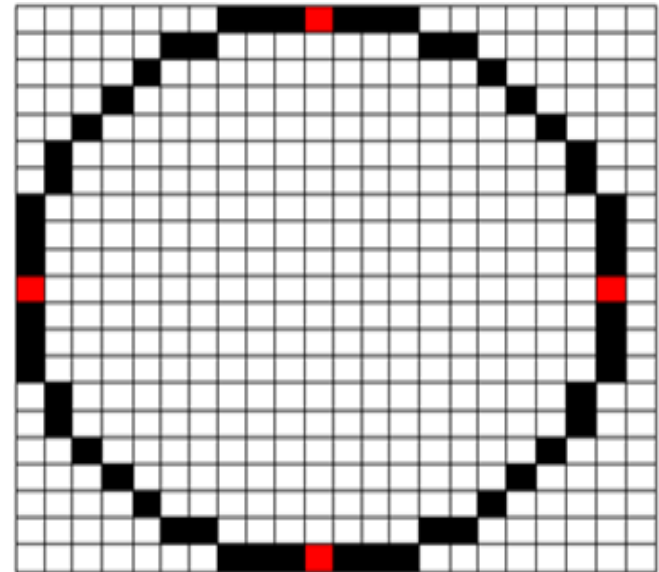
Step-03 is executed similarly until $X_{k+1} \geq Y_{k+1}$

Now, the points of octant-2 are obtained using the mirror effect by swapping X and Y coordinates.

P_k	P_{k+1}	(X_{k+1}, Y_{k+1})
		(0, 10)
-9	-6	(1, 10)
-6	-1	(2, 10)
-1	6	(3, 10)
6	-3	(4, 9)
-3	8	(5, 9)
8	5	(6, 8)
Algorithm Terminates These are all points for Octant-1.		

Octant-1 Points	Octant-2 Points
(0, 10)	(8, 6)
(1, 10)	(9, 5)
(2, 10)	(9, 4)
(3, 10)	(10, 3)
(4, 9)	(10, 2)
(5, 9)	(10, 1)
(6, 8)	(10, 0)
These are all points for Quadrant-1.	

Quadrant-1 (X,Y)	Quadrant-2 (-X,Y)	Quadrant-3 (-X,-Y)	Quadrant-4 (X,-Y)
(0, 10)	(0, 10)	(0, -10)	(0, -10)
(1, 10)	(-1, 10)	(-1, -10)	(1, -10)
(2, 10)	(-2, 10)	(-2, -10)	(2, -10)
(3, 10)	(-3, 10)	(-3, -10)	(3, -10)
(4, 9)	(-4, 9)	(-4, -9)	(4, -9)
(5, 9)	(-5, 9)	(-5, -9)	(5, -9)
(6, 8)	(-6, 8)	(-6, -8)	(6, -8)
(8, 6)	(-8, 6)	(-8, -6)	(8, -6)
(9, 5)	(-9, 5)	(-9, -5)	(9, -5)
(9, 4)	(-9, 4)	(-9, -4)	(9, -4)
(10, 3)	(-10, 3)	(-10, -3)	(10, -3)
(10, 2)	(-10, 2)	(-10, -2)	(10, -2)
(10, 1)	(-10, 1)	(-10, -1)	(10, -1)
(10, 0)	(-10, 0)	(-10, 0)	(10, 0)
These are all points of the Circle.			



Example2: Given the center point coordinates (4, 4) and radius as 10, generate all the points to form a circle

Step-1, Step-2 and Step-3 are already completed in example1.

(X_{k+1}, Y_{k+1})	$(X_{\text{plot}}, Y_{\text{plot}})$
(0, 10)	(4, 14)
(1, 10)	(5, 14)
(2, 10)	(6, 14)
(3, 10)	(7, 14)
(4, 9)	(8, 13)
(5, 9)	(9, 13)
(6, 8)	(10, 12)
(8, 6)	(12, 10)
(9, 5)	(13, 9)
(9, 4)	(13, 8)
(10, 3)	(14, 7)
(10, 2)	(14, 6)
(10, 1)	(14, 5)
(10, 0)	(14, 4)
These are all points for Quadrant-1.	

Quadrant-1 (X,Y)	Quadrant-2 (-X,Y)	Quadrant-3 (-X,-Y)	Quadrant-4 (X,-Y)
(4, 14)	(4, 14)	(4, -6)	(4, -6)
(5, 14)	(3, 14)	(3, -6)	(5, -6)
(6, 14)	(2, 14)	(2, -6)	(6, -6)
(7, 14)	(1, 14)	(1, -6)	(7, -6)
(8, 13)	(0, 13)	(0, -5)	(8, -5)
(9, 13)	(-1, 13)	(-1, -5)	(9, -5)
(10, 12)	(-2, 12)	(-2, -4)	(10, -4)
(12, 10)	(-4, 10)	(-4, -2)	(12, -2)
(13, 9)	(-5, 9)	(-5, -1)	(13, -1)
(13, 8)	(-5, 8)	(-5, 0)	(13, 0)
(14, 7)	(-6, 7)	(-6, 1)	(14, 1)
(14, 6)	(-6, 6)	(-6, 2)	(14, 2)
(14, 5)	(-6, 5)	(-6, 3)	(14, 3)
(14, 4)	(-6, 4)	(-6, 4)	(14, 4)
These are all points of the Circle.			

Example 3

Given a Circle with center at (2, 3) and radius 2, trace the mid point circle drawing algorithm for 4 points:

Hints:

$$p_0 = 1 - r$$

If ($p_k < 0$)

$$p_{k+1} = p_k + 2x_{k+1} + 1$$

Otherwise

$$p_{k+1} = p_k + 2x_{k+1} + 1 - 2y_{k+1}$$

k	x_k	y_k	p_k
0	0	2	-1
1	1	2	2
2	2	1	5
3	2	0	-
4	2	-1	-

Ans:

$$p_0 = 1 - r = -1$$

$$p_1 = p_0 + 2x_1 + 1 = -1 + 2 + 1 = 2$$

$$p_2 = p_1 + 2x_2 + 1 - 2y_2 = 2 + 4 + 1 - 2 = 5$$

decision is to keep y unchanged while incrementing x (1, 2)

decision is to decrement y while incrementing x (2, 1)

decision is to decrement y while incrementing x (3, 0)

With geometrical drawing, you will notice that at this point (3, 0) is outside the circle, because the radius is 2, then we finished this quadrant for this small circle and can just use symmetry (x, -y) to calculate next points without applying the conditions

(2, 0)

(2, -1)

(1, -2)

(0, -2)



Example 4

- Given a Circle with center at (4, 5) and radius 6, trace the mid point circle drawing algorithm for 4 points:

Hints:

$$p_0 = 1 - r$$

If ($p_k < 0$)

$$p_{k+1} = p_k + 2x_{k+1} + 1$$

Otherwise

$$p_{k+1} = p_k + 2x_{k+1} + 1 - 2y_{k+1}$$

- Moving to origin and starting from (0, r) = (0, 6)

$$p_0 = 1 - r = -5$$

$$p_1 = p_0 + 2x_{k+1} + 1 = -5 + (2*1)+1 = -2$$

$$p_2 = p_1 + 2x_{k+1} + 1 = -2 + (2*2)+1 = 3$$

$$p_3 = p_2 + 2x_{k+1} + 1 - 2y_{k+1} = 3 + (2*3)+1 - 2*5 = 0$$

$$p_4 = p_3 + 2x_{k+1} + 1 - 2y_{k+1} = 0 + (2*4)+1 - 2*4 = 1$$